envguard: Eliminating Variable Chaos

Zero-dependency validation for rock-solid configurations.

Rohan S Mirjankar
Roopesh S
Spoorthi T J
Srujan B R

The Configuration Challenge

Environment Variables: A Silent Source of Production Failures

Missing Key Failures

Node.js apps frequently crash with "undefined" errors when essential keys are missing from .env files, causing sudden, unpredictable runtime production outages.

Data Type Confusion

Logic failures occur because environment variables are often parsed as strings when the application logic strictly requires number formats for critical calculations.

Ghost Variable Accumulation

Codebases accumulate unused "ghost" variables in production over time, which increases the overall attack surface and adds significant technical configuration debt.

EnvGuard Three-Layered Defense

EnvGuard resolves configuration chaos with three integrated tools that work throughout the development lifecycle to ensure your environment stays healthy and in sync.

Variable Validation

It validates variables at application startup, ensuring the current environment matches your required schema configurations perfectly.

Codebase Scanner

It analyzes your entire codebase using a CLI scanner to find discrepancies between what is defined and what is actually used in production.

Dev Dashboard

It provides a local UI that gives you a visual green-light for your environment health, ensuring your team stays in sync without manual audits.

Architecture Feature

Application Startup Guarding

The guard() function acts as a gatekeeper, preventing unsafe application starts by enforcing deep validation, strict type coercion for env variables, and immediate character length checks.

Schema Enforcement

Define required fields and specific minLength constraints for sensitive keys like JWT_SECRET.

Type Coercion

Automatically converts string-based environment values into native JS numbers or booleans.

Zero-Schema Mode

Use inferEnv() to automatically detect and coerce types without writing custom schema code.

Unified Reporting

Logs all validation failures at once, allowing for complete configuration fixes in one go.

Automated Environment Variable Analysis with npx envguard

Usage Detection

Detects dot, bracket, and destructuring patterns like process.env.VAR across the project.

Parsing & Stripping

Ignores commented code and handles optional chaining to avoid false positive error results.

Discrepancy Reporting

Identifies unused variables in your .env file and missing variables used within the code.

Invalid Value Check

Flags variables that are currently set to empty strings, null, or undefined values today.

A Lightweight Footprint with Enterprise-Grade Validation

Zero-dependency Architecture

Utilizes native Node.js logic to keep your project lightweight and exceptionally fast.

Flexible Syntax Support

Native support for Dot, Bracket, and Destructuring patterns for environment variables.

Built-in Type Coercion

Includes automated coercion for Numbers and Booleans to ensure correct data types.

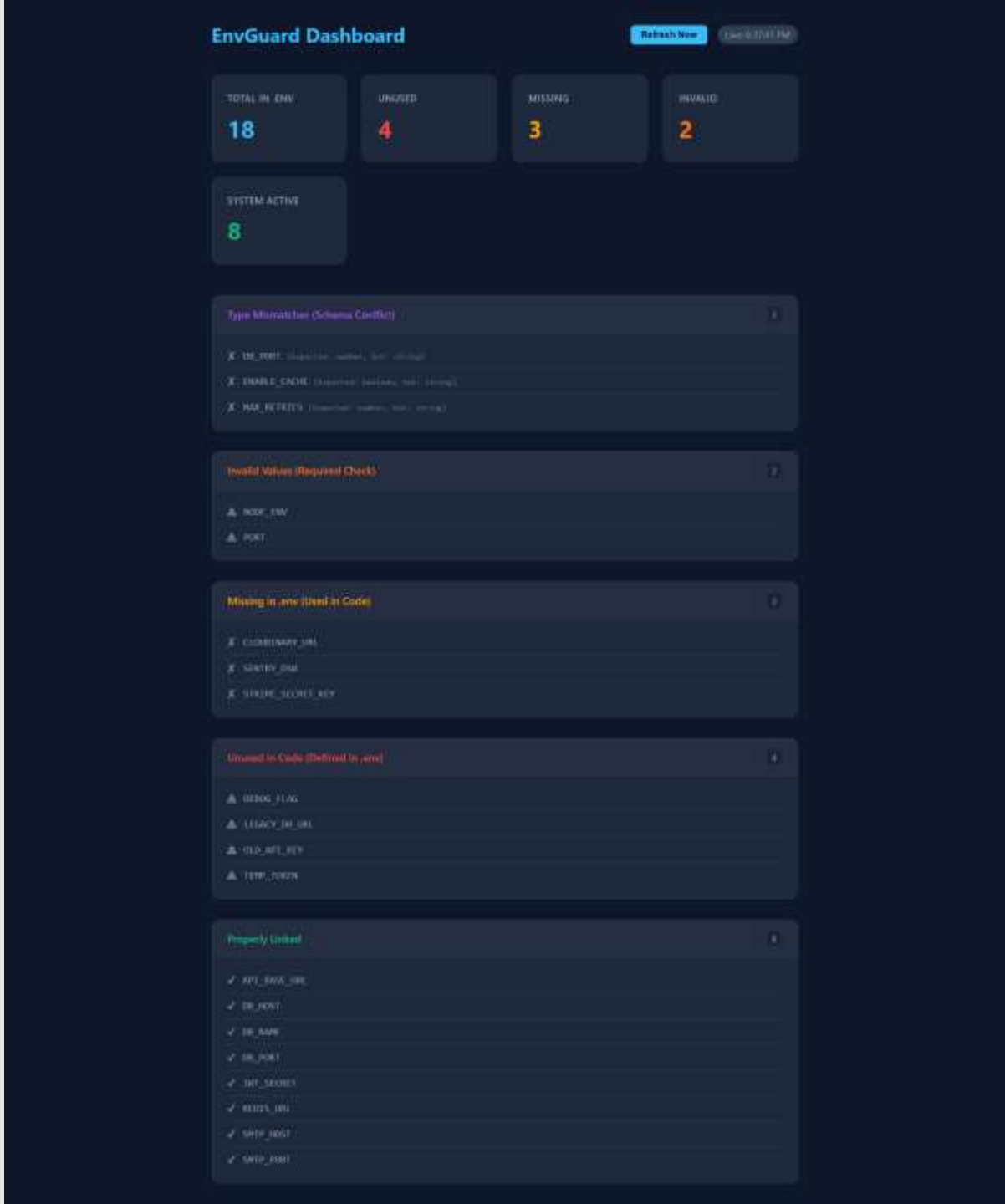
Unified Error Logging

Consolidates all validation failures into a single, comprehensive error log stream.

Real-time Local Dashboard

Provides a dedicated Local Dashboard with real-time polling to monitor variable status.

WEBSITE PREVIEW



Achieve Envguard Stability

By moving from manual checks to automated validation and analysis, engineering teams can eliminate one of the most common causes of deployment failure. envguard turns environment variables from a source of chaos into a source of truth.

Catch issues earlier in the development lifecycle with fast feedback and consistent validation—before they reach production.

Empower teams with confidence and clarity through standardized rules and transparent reporting across all environments.